Georgia Tech

USER AND KERNEL THREADS

ALEXEY TUMANOV

School of Computer Science
faculty.cc.gatech.edu/~atumanov
atumanov@gatech.edu

CREATING THE NEXT®

Lab3 – Released !!

Start early… Please.

Scheduling + Threading library implementation

Timeline: 5 weeks for lab3 …. BUT

- 1 week spring break (week of 3/23)
- 1 week midterm prep (week of 3/03-3/10)
- 3 weeks remaining , bigger lab than lab2 (3 weeks)!

Midpoint checkpoint:

- *checkpoint 1*: 3/20 (parts: 1&2, tests 1-21)
- autograder calculated score logged on each submission
- custom scheduler design question! Beat the system!

bonus

bonus

Exam 1: SecA-B: Tue, Mar 10, 6:30-8pm

- In-class (supervised lab), on canvas, proctored
- Topic coverage: everything up to and *excluding security*, e.g.
  - X86 architecture and boot (lec3)
  - Isolation, protection (lec4)
  - kernel organization (lec5)
  - VMM (lec6-9)
  - Interrupts, concurrency (lec10-11)
  - Scheduling/waiting (lec12-13)
  - User and kernel threading (Lec 14)
- Lecture material/content will be tested

LOGISTICS

- Withdrawal deadline: ***Wed, Mar 18th***
- Exam 1 **will** be graded by then
- Lab3 checkpoint : 3/20, target 3/18 for better mid-point grade feedback
- Topics ***after*** exam 1:
  - Security
  - OS & Networking
  - File Systems
  - Ordering and Atomicity (in File Systems)
  - Distributed Systems

WHY THREADING

- Why do I want threading? (General discussion)
  - Are there other benefits than performance?
    - I can get performance with multi-processing.
    - *Design*: threading vs multi-processing vs event-based
- Threading *benefits*
  - Parallelism with shared memory resources
  - An Abstraction (!) to share a resource (e.g. address space)
    - A way to multi-process/manage concurrency with shared memory resources
- Threading *challenges*
  - synchronization

THREADING

- What even is a thread?
  - Two independent entities, doing computation, sharing memory
  - What *alternatives* are there to threads?
    - What if I didn't want threads in my system?

- Additional threading benefits:
  - Think single core multi-threading? (used threads before 2004)
    - Benefits from threading w/o perf benefit??
  - *Organization*, processing asynchronous events
    - E.g. UI thread + background process
    - Simultaneity, abstraction of concurrency/simultaneity of action
  - Threads give us the *abstraction of concurrency*

ABSTRACTION OF CONCURRENCY

- Threading is a ***mechanism***!
  - Of what?
- ***Abstraction*** of concurrency:
  - Ability to seamlessly execute independent threads of execution at the same time
- ***Mechanisms of Concurrency:***
  - Threads
  - Multi-processing
  - Event-based systems! (single thread, single process)
    - Voluntarily preemptive mechanism

Takeaway: gain a convenient abstraction of concurrency

DESIGN SPACE EXPLORATION: MECHANISMS OF CONCURRENCY

- **Multi-processing**
  - Processes that share compute resources, but not memory resources
  - Key property guaranteed: memory space isolation!
- **Threads**
  - Ask a disruptive question: what if we break this isolation guarantee, deliberately
  - Key property: shared memory resources, by design
- **Event-based systems**
  - Single process, single thread
  - Voluntarily preemptive

USER AND KERNEL THREADS

Georgia Tech

THREAD DEFINED

"**thread** of execution is the smallest sequence of programmed instructions that can be managed independently by a scheduler" [2]

THREAD DEFINED AGAIN:

"a thread, sometimes called a lightweight process, is a basic unit of CPU utilization; it comprises a thread ID, a program counter, a register set, and a stack" [1]

Main property: Shared memory

USER AND KERNEL THREADS

Benefits [2]

Responsiveness: Especially for interactive applications

- Provide the abstraction of concurrency

Resource Sharing: Default definition of process defines separate address space, default thread shares memory and resources

Explain!

Economy: Creation / destruction much faster, less ***overhead***, etc.

Multiprocessors: Perhaps achieve true parallelism vs single-processor illusion

THREADING: DESIGN + IMPLEMENTATION

Georgia Tech

What ***design tradeoffs*** do we make when supporting threads?

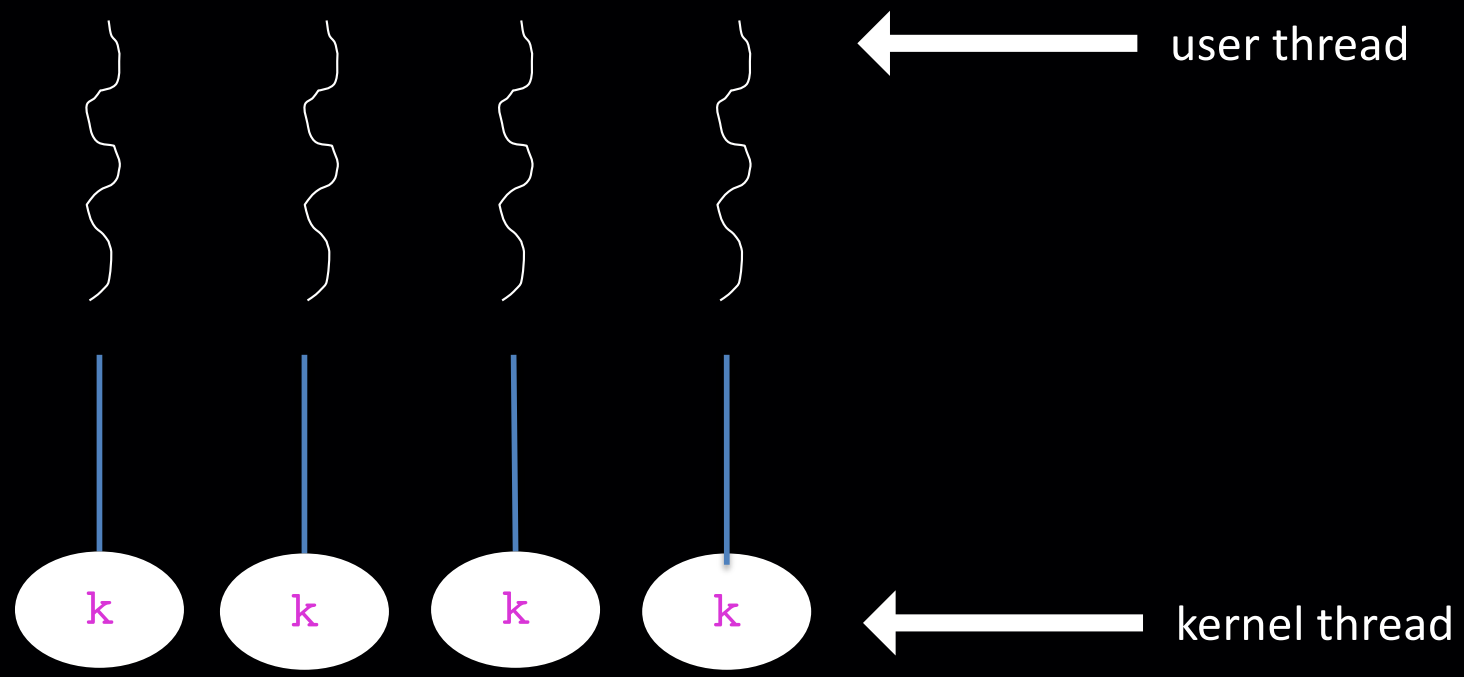
- How do we best structure and enable *threading* support?
- What is the real difference between a *thread* and a *process*? Does there really need to be one?
- What is the interface for thread creation / destruction / management?
  - *Creating* threads
  - *Destroying* threads
  - *Managing* threads

Georgia Tech

Design Choices – **One-to-One** (AKA 1:1) - "Kernel Threading"

Attributes:

- Improved concurrency
- Can run in parallel on multiprocessor
- Additional overhead to create kernel thread for every user thread

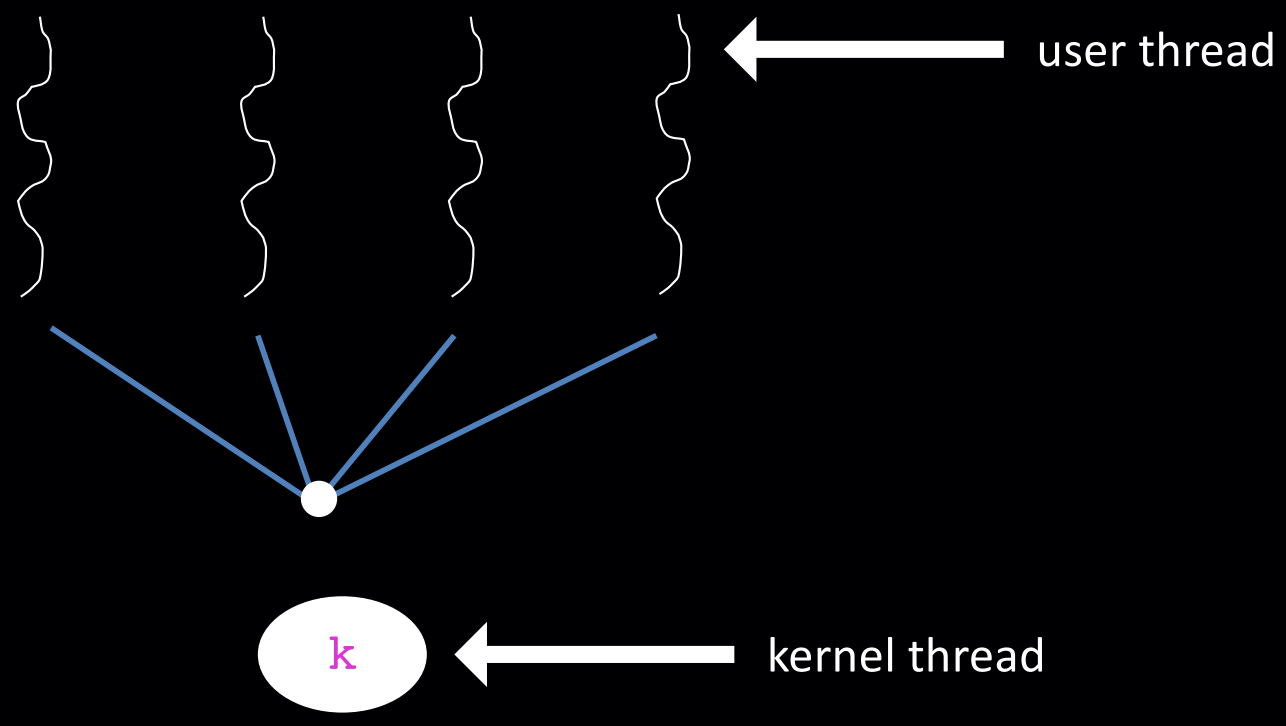


Georgia Tech

Design Choices – Many to One – "User Space Threading"

Attributes:

- Map many user threads to one kernel thread
- Simple, efficient
- True concurrency is difficult (ex. One thread blocks all)
- Only one thread can access kernel at a time, so no parallelism on multiprocessors



USER-SPACE THREADING LIBRARY

- How does a ***user-space*** threading library work? (sched deck)
  - Pretty much the same way xv6 manages “kernel” threads.
  - Recall, the kernel doesn’t use any hardware protections for threading
    - **Swtch** – does context switch
    - Scheduler chooses what to run.
    - Concurrency primitives are built from atomics and queues (can build in user-space)

`Context switch could happen in user space`

- Are there any primitives that the user-space needs the kernel for?
  - (Think, what abstractions does the kernel provide?)

USER-SPACE THREADING LIBRARY

Georgia Tech

- Why does a user-space threading library make sense:
  - Key *observation*: The threading mechanism is designed with an implicit assumption of ***cooperation*** between units of execution
  - *Implication*: cooperative scheduler is sufficient
  - User-space cooperative scheduler can switch between user-space threads without relying on preemption mechanism available in/to the kernel!
  - Threads are cooperative
  - Synchronization primitives can be built with user-space atomics and concurrency primitives
  - No privileged instructions needed from the kernel
- Thread context, stacks, register state available in user space
- Can switch stacks in user space!

CONCURRENCY VS PARALLELISM

Georgia Tech

Concurrency

- Is an abstraction
- Has multiple mechanisms
- An illusion of making concurrent progress on multiple execution streams
- Can be achieved on :
  - A single core
  - Single process
  - Single thread
  - => event based system

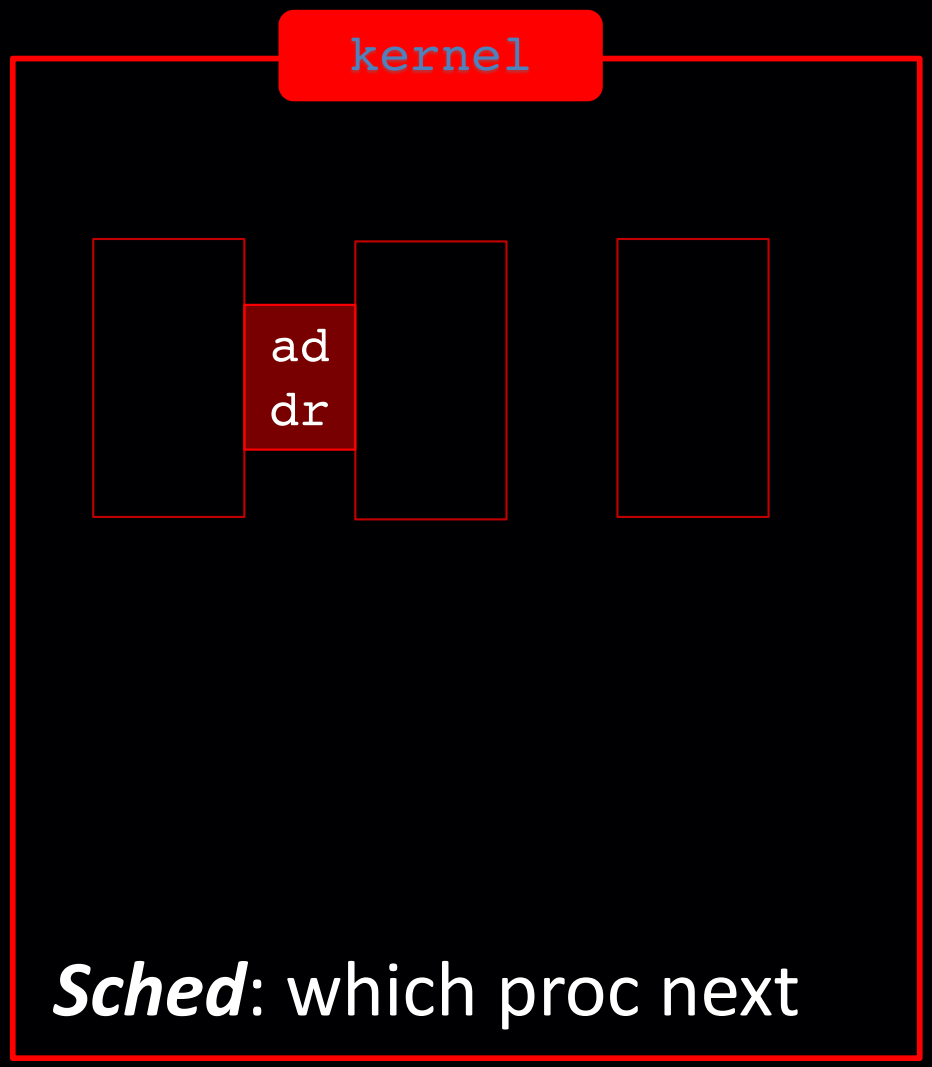
Do we have *concurrency*?

Do we have *parallelism*?

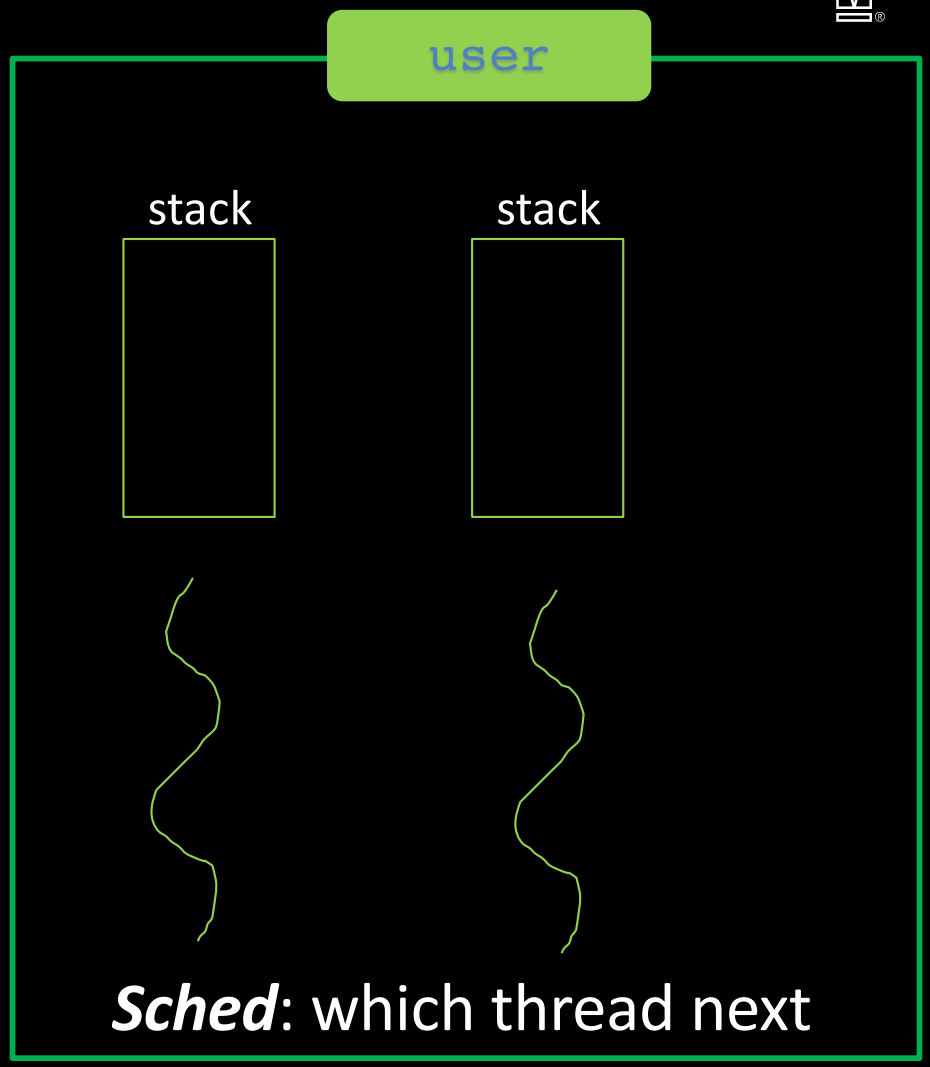
Parallelism (True Parallelism)

- An act of executing something truly in parallel
- *Simultaneous* execution of multiple execution streams
- Concurrency mechanisms (e.g. threading) can be used to achieve parallelism (ability to execute multiple things in parallel)

KERNEL VS USER-SPACE THREADING

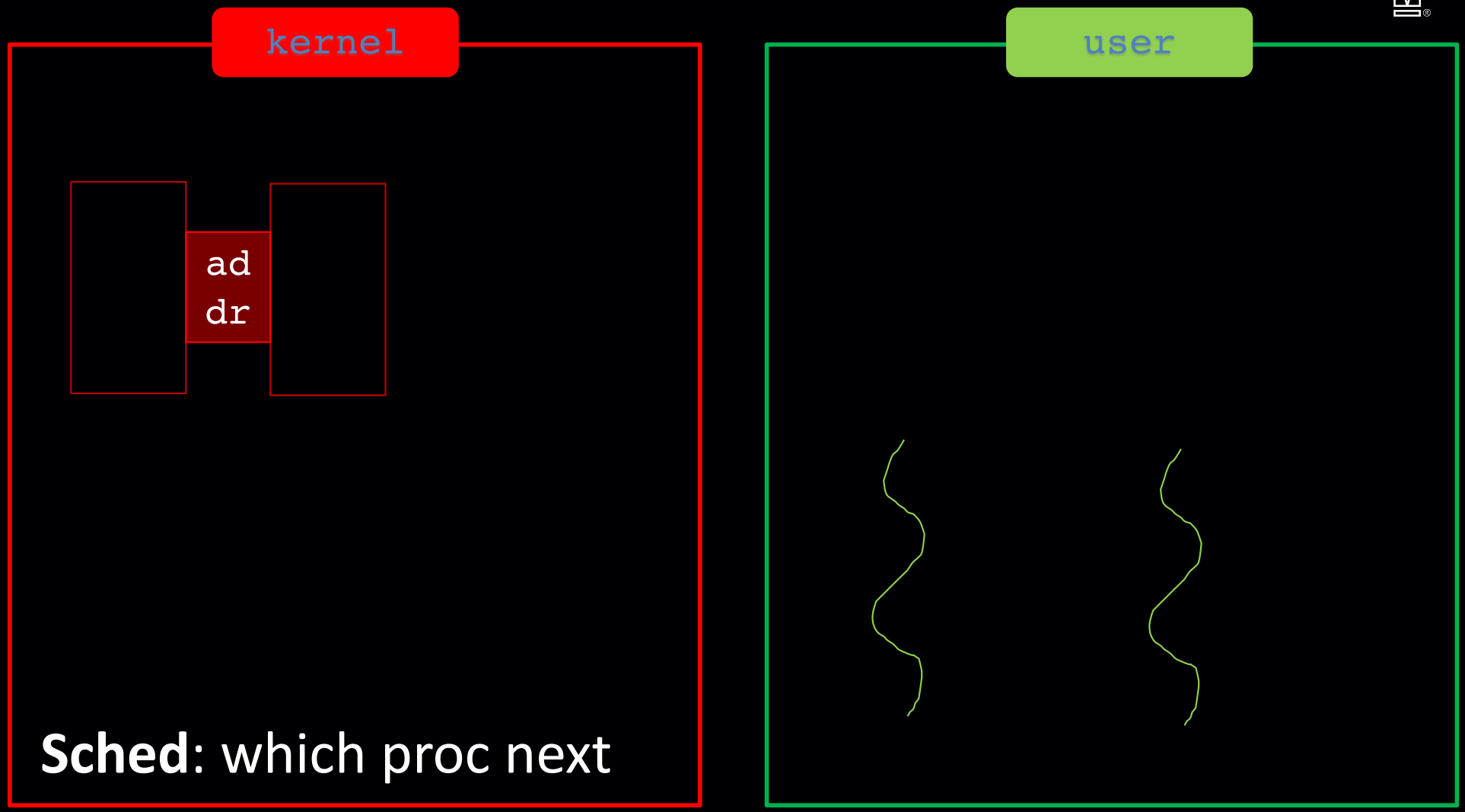


Many procs



1 proc

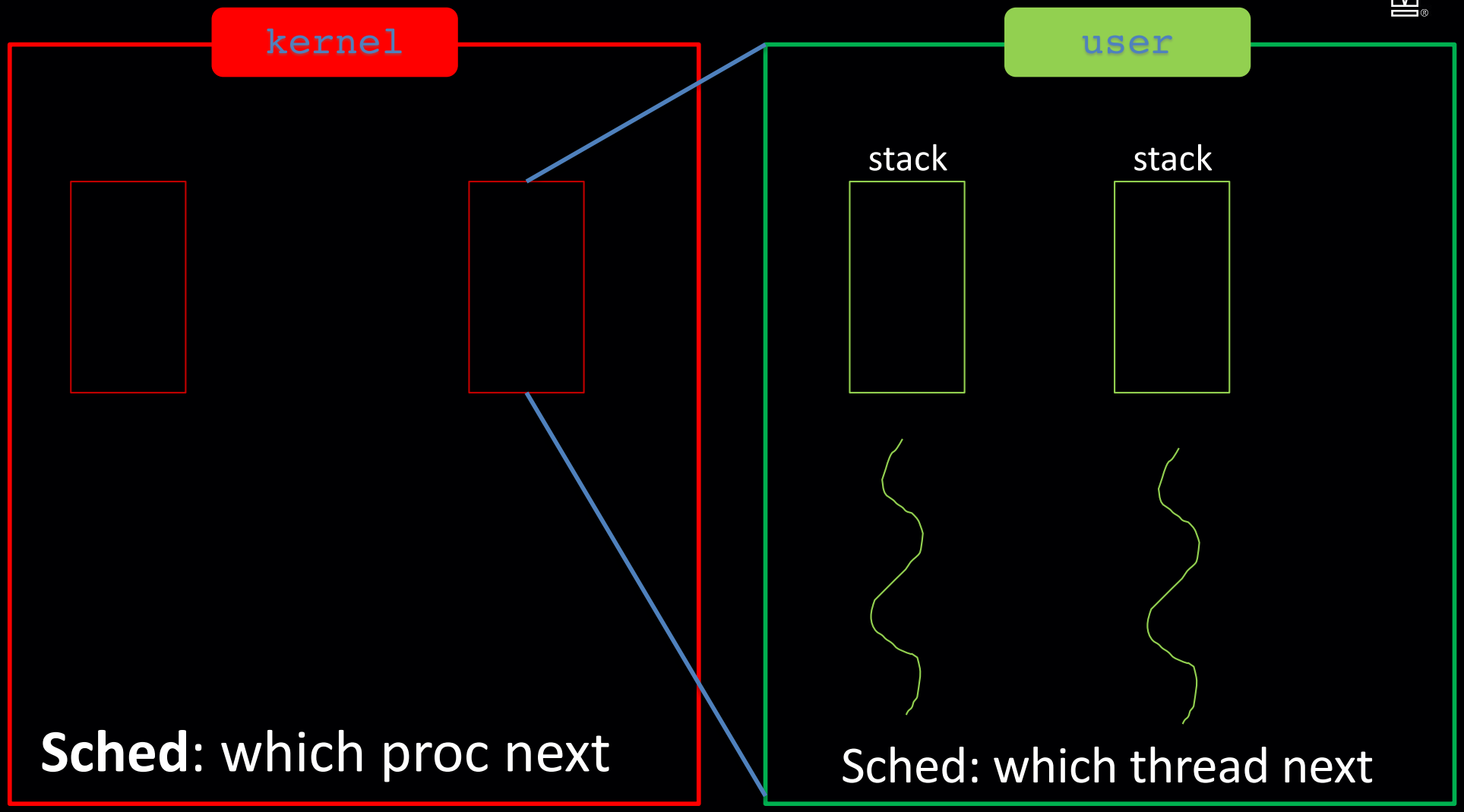
KERNEL THREADING



Sched: which proc next

Many procs

USER-SPACE THREADING



Many procs

1 proc

Semantic gap

ADVANTAGES / DISADVANTAGES

- What are the advantages of kernel space threading over user-space threading?

- What are the advantages of user-space threading over kernel space threading?

interesting

Breakout discussion.

KERNEL VS. USER-SPACE THREADING

Georgia Tech

Kernel-space:

- Parallelism
- Preemption possible
- More information available to the kernel
- Ability to control resource utilization
- Thread-aware process sched

User-space:

- Semantically-aware scheduling (application awareness)
- Context switching overhead
- Cache locality
- Lighter weight than kernel threading
- Overhead of thread creation

USER SPACE THREADING: DO WE GET TRUE PARALLELISM?

Does user-space threading actually get parallelism?

Thread 1:

```
do_work();   // long running
sleep(100);  // sleep

do_work();  // long running
```

Thread 2:

How will user-space know to yield before read?

```
// Waits for a very long time
read(network, buf);
```

cpu1

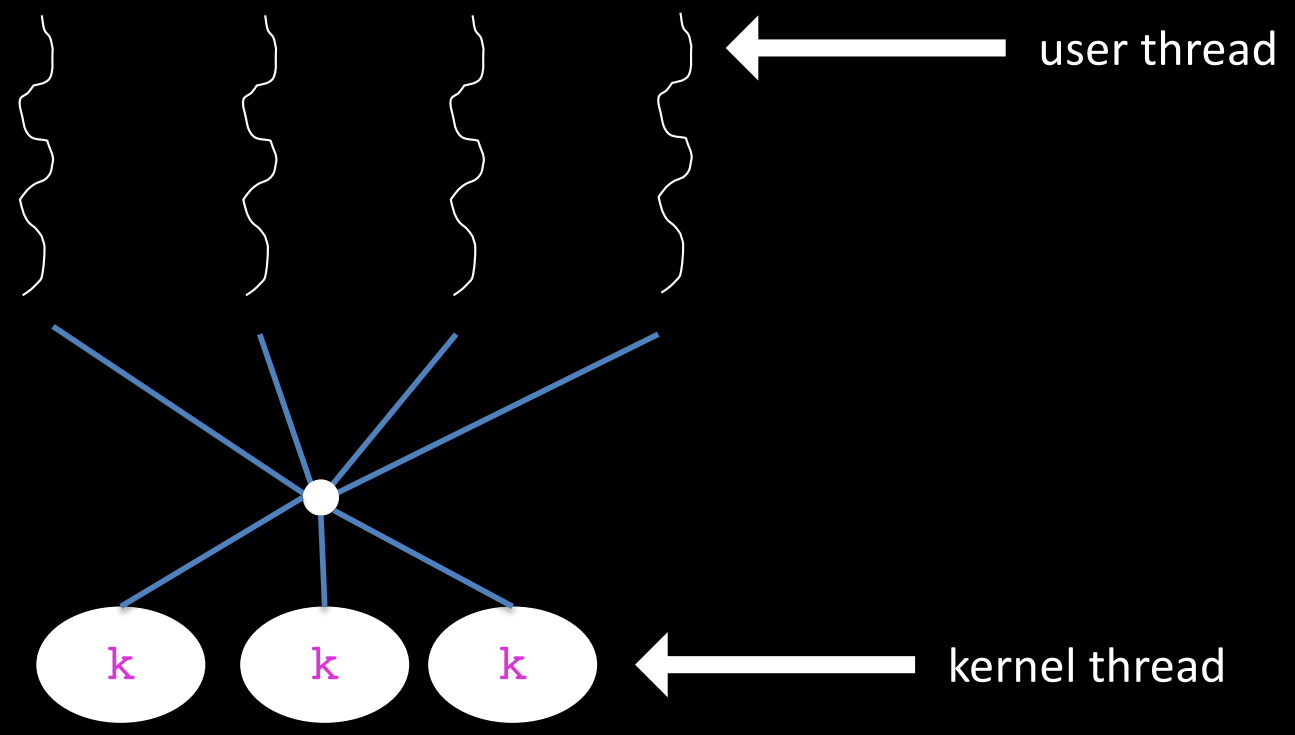
KERNEL THREADING: DO WE GET TRUE PARALLELISM?

Georgia Tech

Thread 1:	Thread 2:
do_work(); // long running	
sleep(100); // sleep	
	// Waits for a very long time
	read(network, buf);
do_work(); // long running	
cpu1	cpu2

HYBRID THREADING (M-N)

Design Choices – Many to Many (AKA M:N, AKA Hybrid Threading)



HYBRID VS USER-SPACE

Advantages of M:N

- User-space threads can run in **parallel**
- Fewer context switches
- Light weight user-space threads (small user space thread cost)

Disadvantages?

- Still subject to kernel thread stalls causing user space thread **blocking**
- Still not cooperative use of CPU

MODERN THREADING LIBRARIES

- Mostly *kernel* threading (pthreads)
- However, many operations in user-space:
  - "*futex*" syscall allows mostly user-space concurrency primitive construction
    - Essentially kernel sleep/wakeup mechanism
    - Allows user-space to do locking, but ask kernel for sleep/wakeup
- Trade-off of user/kernel efficiency
  - Kernel manages IO sleeping / process sleeping / scheduling
  - User-space manages most concurrency primitives (lock, cv, semaphore)

USER AND KERNEL THREADS: XV6

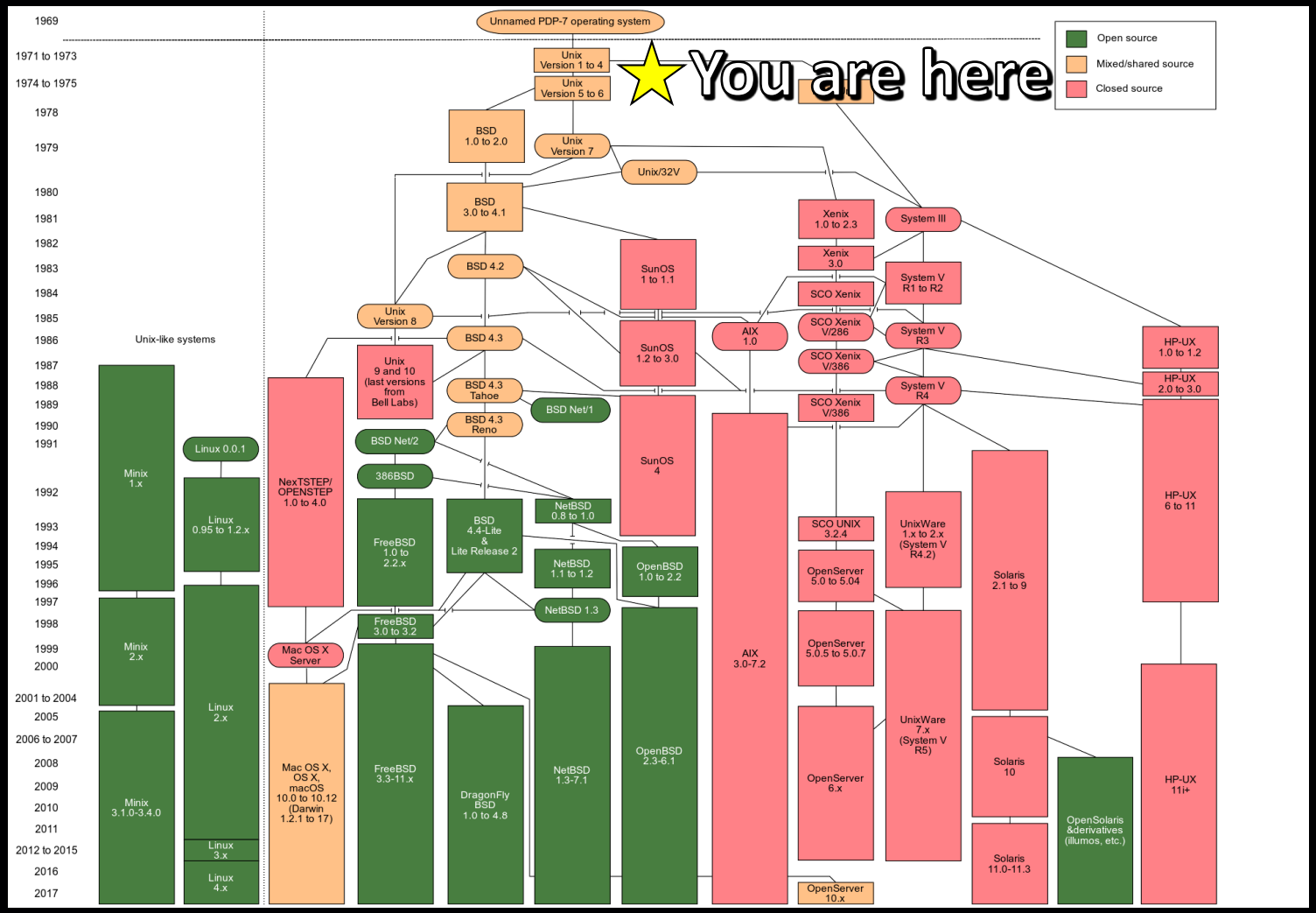
Where do we stand in **xv6**?

- Kernel stack – Each process has own kernel stack
- User threads – There are none

Scheduler has no awareness (No SMP support)

USER AND KERNEL THREADS

A historical UNIX perspective



By Eraserhead1, Infinity0, Sav_vas - Levenez Unix History Diagram, Information on the history of IBM's AIX on ibm.com, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=1801948

USER AND KERNEL THREADS

Where do we want to be? (The **Goal**)

Let's say we want to implement kernel thread support in xv6 using a 1:1 model

- Allow threads to be scheduled
- Retain performance advantages of threads
  - Share memory, perf advantages of kernel threading
- Synchronization primitives
  - Waiting (waitpid, thread_wait)
  - Spinlocks (init, acquire, release)
  - Mutexes: (park/setpark/unpark , mutex init/acquire/release)
  - Conditional variables (cond_wait, cond_signal)

lab3

USER AND KERNEL THREADS

How do we get there?

How do I create a thread?

- Add “clone()” system call - create new process that shares pgdir
- Add a user API for creating threads
- Locking primitives

lab3

USER AND KERNEL THREADS: TRADEOFFS

Recap:

Kernel threading vs user-space threads?

- Multiple user threads which must share a single kernel thread are inherently limited
  - Ex. Thread blocks process during blocking system call
- Poor scheduling decisions due to lack of awareness regarding user threads (semantic awareness or ``semantic gap'')
- True parallelism – each process only has 1 CPU otherwise

USER AND KERNEL THREADS

Georgia Tech

Another idea: Scheduler Activations (circa 1991) [10]

One way that M:N scheduling has been realized

Concept: Introduce a communication mechanism between kernel space and user space

1. Kernel notifies address space thread scheduler of kernel events affecting that address space-– application gains knowledge of scheduling state (often using "upcall")
2. User space can notify kernel

"A scheduler activation vectors control from the kernel to the address space thread scheduler on a kernel event; the thread scheduler can use the activation to modify user-level thread data structures, to execute user-level threads, and to make requests of the kernel"

Concept was implemented in NetBSD but ultimately abandoned

Also attempted with Linux (2.4X) (Patch + Marcel library)

USER AND KERNEL THREADS

Current Threading Support by Operating System (2018 examples)

Operating System	Introduced	Model	Comment
Minix		N/A	It's complicated
Linux (LinuxThreads)	1996	1:1	
Linux (NPTL)	2003	1:1	
macOS / iOS[7,8]	~2000	1:1ish	Built on Mach Threads, BSD layer
FreeBSD[3]	2003	M:N or 1:1 (selectable)	1:1 is default / recommended as of 2006
DragonFly BSD	2007	1:1	
NetBSD[4]	2009	1:1	1:1 since 5.0 (circa 2009)
OpenBSD	2012?	1:1	Just recently!
Solaris	2000	M:N then 1:1	
HP-UX	1997	M:N or 1:1	
Windows	?	1:1	

CLONE

In Linux, clone() is a wrapper that supports thread or process creation, via the common construct known as a “task”:

- Every task is a schedulable entity
- Some tasks may share more or fewer resources
  - fork() gets own memory space (new process, still using clone())
  - clone() where memory space is ***shared*** is a new thread

USER AND KERNEL THREADS

Linux: clone() system call

```
long clone(unsigned long flags, void *child_stack,
           void *ptid, void *ctid,
           struct pt_regs *regs)
```

Linux: clone() library function

```
int clone(int (*fn)(void *), void *child_stack,
          int flags, void *arg, ...
          /* pid_t *ptid, struct user_desc *tls, pid_t *ctid
*/ );
```

- On Linux, clone() underpins both threads and processes (fork())

USER AND KERNEL THREADS

A Simpler Interface for xv6 (lab3)

```
int clone(void *stack, int size)
```

Implementation? Similar to fork(), except:

- Reuse the parent's address space entirely (no COW)

USER AND KERNEL THREADS

POSIX Threads (pthread) interface

```
int pthread_create(pthread_t *thread,
                   const pthread_attr_t *attr,
                   void *(*start_routine) (void *),
                   void *arg);
```

USER AND KERNEL THREADS

A Simpler Interface for xv6

```
thread_create(void *(*start_routine)(void*),
              void *arg)
```

USER AND KERNEL THREADS

Georgia Tech

REFERENCES

[1] Avi Silberschatz, Peter Baer Galvin, and Greg Gagne. 1999. Applied Operating System Concepts. John Wiley & Sons, Inc., New York, NY, USA.

[2] https://en.wikipedia.org/wiki/Thread_(computing)

[3] https://www.freebsd.org/doc/en_US.ISO8859-1/articles/linux-emulation/inside.html Pulled 26 March 2018

[4] http://www.netbsd.org/~rmind/pub/netbsd_5_scheduling_apis.pdf

[5] http://www.netbsd.org/changes/changes-5.0.html#newlock2

[6] http://mirror.informatimago.com/next/developer.apple.com/technotes/tn/tn2028.html

[7] http://osxbook.com/book/bonus/ancient/whatismacosx/arch_xnu.html

[8] Levin, Jonathan, “Mac OS X and iOS Internals To the Apple’s Core”. 2003

[9] https://www.youtube.com/watch?v=ggnFoDqzGMU

[10] https://homes.cs.washington.edu/~bershad/Papers/p53-anderson.pdf

[11] http://www.informit.com/articles/article.aspx?p=25075

[12] https://news.ycombinator.com/item?id=4729150

[13] https://www.openbsd.org/52.html

[14] http://www.kegel.com/c10k.html#1:1

[15] https://www.codeproject.com/Articles/662735/Internals-of-Windows-Thread